ShopSphere — E-commerce Shopping Funnel (DBMS Project Report)

1. Project Title

ShopSphere: An E-commerce Shopping-Funnel Database with Behaviour Analytics

Database Management Systems (DBMS) course project, October 2026. Built with MySQL 8 and Django.

Course: 22AIE303 Database Management Systems.

GitHub repository: https://github.com/Bhavesh-max-jpj/E-commerce-project. It holds the source code, the database scripts (`database/`), documented queries, tests, screenshots and this report.

2. Team Members

#	Name	Roll number
1	Nagireddy Bhavesh Reddy	CB.SC.U4AIE24034
2	Bachu Harini	CB.SC.U4AIE24010
3	Kallam Pratheep Reddy	CB.SC.U4AIE24021
4	Ravala Gnanasri	CB.SC.U4AIE24064

3. Introduction and Problem Statement

ShopSphere is an online clothing and footwear store whose database records every step a shopper takes, from viewing a product to paying and reviewing it, so the store can see where customers drop off. It runs on Django with a MySQL 8 database (`shopping_funnel`) holding 2,905 products from 16 sellers.

Problem statement.

- Online stores record the orders they receive, but not the shoppers who view products and leave without buying.
- Without stage-by-stage data, a store cannot tell whether customers drop off after viewing, in the cart, at checkout or at payment.

- Rules such as stock limits are often checked only in the web application, so wrong or inconsistent data can still reach the database.
- The store needs conversion and drop-off rates by product, category, brand, seller and device to decide which step to fix.

Approach. Each customer action is written to a `funnel_events` table. MySQL triggers, views and stored procedures then turn those events into a six-stage funnel (VIEW → CART → CHECKOUT → ORDER → PAYMENT → REVIEW) with conversion rates, drop-off, cart abandonment and breakdowns by product, category, brand, seller and device.

Main finding on the demo data. About 57% of visitors add something to the cart, but mobile shoppers complete only about 44% of their carts against about 60% on desktop, so mobile checkout is the weakest step.

Tech stack

Layer	Technology	Used for
Database	MySQL 8.0, MySQL Workbench	13 tables, constraints, 4 triggers, 9 views, 3 stored procedures
Backend	Python 3.12, Django 5.2 / 6.x	Pages, logins, checkout and payment transactions
Database driver	PyMySQL 1.1	Connects Django to MySQL using parameterised queries
Frontend	HTML templates, CSS, JavaScript	Customer, seller and admin pages; 3D effects and dark mode
Charts	Chart.js 4.4 (bundled)	Funnel and daily charts on the analytics page
Dataset	products.csv, sellers.csv, product photos	2,905 products from 16 sellers
Tools	VS Code, Git, GitHub	Development and version control

4. Conceptual and Logical Database Design

The conceptual model has 13 entities in three groups: the catalogue, the sales process and shopper behaviour. Each entity maps to exactly one table, and each one-to-many relationship becomes a foreign key on the "many" side.

Entities.

Group	Entity	Description	Identifier
Catalogue	Seller	A business that lists products	seller_id
Catalogue	Product	One item for sale, with price and stock	product_id
People	Customer	A registered shopper	customer_id
People	Admin	A store administrator	admin_id
Sales	Cart	A product waiting in a customer's cart	cart_id
Sales	Order	One purchase by a customer	order_id
Sales	Order item	One product line within an order (weak entity of Order)	order_item_id
Sales	Payment	A payment attempt for an order	payment_id
Sales	Review	A customer's rating of a product	review_id
Behaviour	User session	One login visit, with device type	session_id
Behaviour	Product view	A product page opened in a session	view_id
Behaviour	Cart event	A change to a cart (add, increase, decrease, remove)	event_id
Behaviour	Funnel event	A customer reaching a funnel stage	event_id

Relationships and cardinalities.

Relationship	Cardinality	Participation
Seller *lists* Product	1 : N	A product may have no seller (optional)
Customer *places* Order	1 : N	Every order has exactly one customer
Order *contains* Product, through Order item	M : N	An order has at least one item
Order *is paid by* Payment	1 : N	Failed attempts allowed; at most one completed
Customer *reviews* Product	M : N, through Review	Rating 1–5

Relationship	Cardinality	Participation
Customer *keeps* Product *in* Cart	M : N, through Cart	One row per customer and product
Customer *starts* User session	1 : N	
User session *includes* Product view	1 : N	
Cart *has history in* Cart event	1 : N	History kept when the cart row is deleted
Customer *reaches stage in* Funnel event	1 : N	Optional links to session, product and order

5. Normalization and Functional Dependencies

Functional dependencies per table.

Table	Functional dependencies	Candidate keys	Normal form
sellers	seller_id → all; email → all	seller_id, email	BCNF
products	product_id → seller_id, gender, category, sub_category, product_type, colour, usage_type, product_title, image, image_url, brand, price, stock_quantity	product_id	BCNF
customers	customer_id → all; email → all	customer_id, email	BCNF
admin	admin_id → all; username → all; email → all	admin_id, username, email	BCNF
cart	cart_id → all; (customer_id, product_id) → cart_id, quantity, added_date	cart_id, (customer_id, product_id)	BCNF
orders	order_id → customer_id, order_date, total_amount, order_status, shipping_addr, delivery_date	order_id	BCNF

Table	Functional dependencies	Candidate keys	Normal form
order_items	order_item_id → order_id, product_id, quantity, unit_price, subtotal; (quantity, unit_price) → subtotal	order_item_id	2NF (subtotal stored on purpose)
payments	payment_id → all; transaction_id → all	payment_id, transaction_id	BCNF
reviews	review_id → customer_id, product_id, rating, review_text, review_date	review_id	BCNF
user_sessions	session_id → customer_id, start_time, end_time, device_type	session_id	BCNF
product_views	view_id → customer_id, product_id, session_id, view_timestamp	view_id	BCNF
cart_events	event_id → customer_id, product_id, cart_id, event_type, event_timestamp	event_id	BCNF
funnel_events	event_id → customer_id, session_id, product_id, event_stage, event_timestamp, order_id	event_id	BCNF

6. Advanced SQL Implementation and Query Design

The database protects its own data. Bad values are refused by constraints, stock and funnel events are handled by triggers, and reports come from views and stored procedures. All of it lives in the numbered SQL scripts in the repository's `database/` folder (`01_schema.sql` to `09_delete_rules.sql`). The 24 documented complex queries are in `database/queries/complex_queries.sql`, and 16 SQL validation tests are in `database/tests/validation_tests.sql`.

Constraints (17 CHECK, 6 UNIQUE, plus all foreign keys).

Rule	Where
Quantity above 0; price, stock and totals 0 or more	cart, order_items, products, orders, payments
`subtotal = quantity × unit_price`	order_items
Rating 1–5; seller rating 0–5	reviews, sellers
Only valid statuses (Pending, Paid, Shipped, Delivered, Cancelled; Completed, Failed, Refunded)	orders, payments
Only valid stages and cart event types	funnel_events, cart_events
Email must look like an email; gender from a fixed list	customers, sellers
One cart row per customer and product; unique transaction ids and emails	cart, payments, customers, sellers
Cart history is kept when a cart row is deleted (`ON DELETE SET NULL`)	cart_events → cart

Triggers.

Trigger	When	What it does
`trg_order_items_before_insert`	before an order line is saved	Locks the product row (`FOR UPDATE`) and raises `SIGNAL 45000` if stock is too low
`trg_order_items_after_insert`	after an order line is saved	Reduces `stock_quantity`
`trg_payments_after_insert`	after a successful payment	Writes a PAYMENT funnel event for each product in the order
`trg_reviews_after_insert`	after a review	Writes a REVIEW funnel event

Views. `vw_funnel_summary`, `vw_product_funnel`, `vw_category_funnel`, `vw_brand_funnel`, `vw_daily_funnel`, `vw_device_funnel`, `vw_cart_abandonment`, `vw_seller_sales` and the helper `vw_customer_product_reach`.

Stored procedures.

- `sp_funnel_between(start, end)`: the funnel for any date range. It is used by the analytics page's date filter.

- `sp_customer_journey(customer_id)`: one customer's steps in time order.
- `sp_top_products(n)`: best sellers by revenue from paid orders.

Transactions. Checkout runs inside one `transaction.atomic()` block: funnel events, the order, its lines and emptying the cart. If the stock trigger refuses any line, everything rolls back and the shopper sees "Not enough stock for …". Payment locks the order row so a double click cannot pay twice. Cancelling returns the stock and marks the payment Refunded in one transaction.

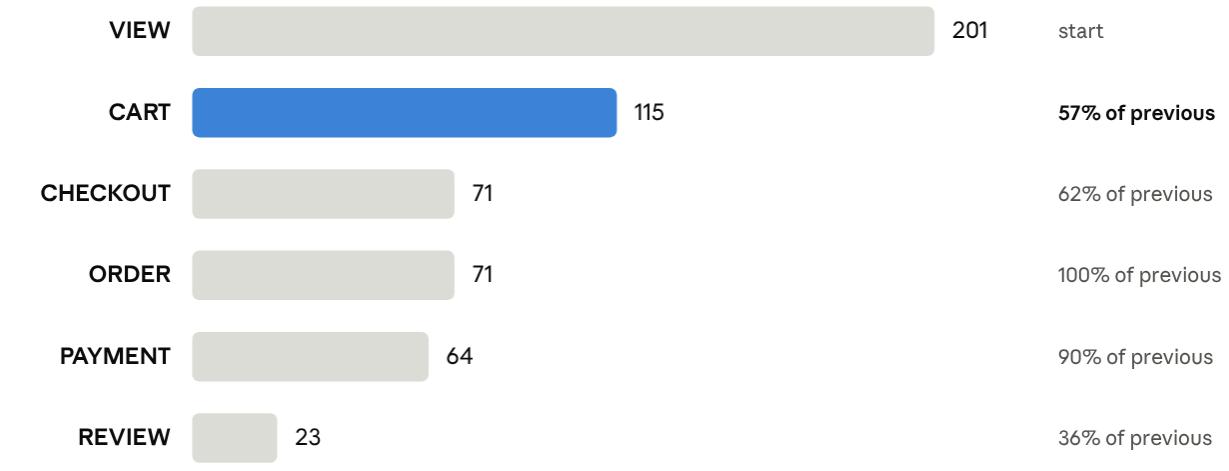
Funnel analytics query design

The funnel counts, for each stage, how many distinct customers got at least that far. On the demo data the biggest loss is the first step: 43% of visitors never add anything to the cart.

Cumulative rule. A customer who reached a later stage also counts in every earlier one (VIEW ⩾ CART ⩾ CHECKOUT ⩾ ORDER ⩾ PAYMENT), because nobody can pay without viewing and carting. In SQL, `MAX(FIELD(event_stage, 'VIEW', 'CART', 'CHECKOUT', 'ORDER', 'PAYMENT'))` finds each customer's furthest stage. Old orders placed before tracking existed therefore still give a sensible funnel. REVIEW is counted separately.

43% of visitors leave before adding anything to the cart

Customers reaching each stage, cumulative rule, demo data (test run, 14 Oct 2026)



REVIEW is counted on its own: a review does not prove a purchase.

vw_funnel_summary on the test copy · 200 demo shoppers + test orders, 14 Oct 2026

Other findings on the same data.

- **Device:** mobile sessions turn 44% of carts into orders, against 60% on desktop and 67% on tablet. Mobile checkout is the weakest step.
- **Category:** footwear converts 54% of carts to payments, apparel 47%.

- **Abandonment:** 137 customer-and-product pairs were carted and never paid. `vw_cart_abandonment` lists them for follow-up.

Key SQL code

The listings below are taken from the project's scripts (`database/04_triggers.sql`, `database/05_views_procedures.sql`, `database/08_cumulative_funnel.sql`). They show a trigger with locking and an error signal, a cumulative view, a stored procedure, and analysis queries using joins, grouping, subqueries and window functions.

Trigger: refuse an order line when stock is too low (`database/04_triggers.sql`)

```
CREATE TRIGGER trg_order_items_before_insert
BEFORE INSERT ON order_items
FOR EACH ROW
BEGIN
    DECLARE v_stock INT;
    DECLARE v_title VARCHAR(255);
    DECLARE v_msg   VARCHAR(255);

    IF NEW.quantity <= 0 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Quantity must be greater
than 0';
    END IF;

    SELECT stock_quantity, product_title INTO v_stock, v_title
      FROM products WHERE product_id = NEW.product_id
       FOR UPDATE;                        -- lock the row: two buyers cannot
take the last item

    IF v_stock IS NOT NULL AND v_stock < NEW.quantity THEN
        SET v_msg = CONCAT('Not enough stock for "', LEFT(v_title, 120), '":
only ', v_stock, ' left');
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = v_msg;
    END IF;
END
```

Trigger: record the PAYMENT stage automatically (`database/04_triggers.sql`)

```
CREATE TRIGGER trg_payments_after_insert
AFTER INSERT ON payments
FOR EACH ROW
BEGIN
    IF NEW.payment_status IN ('Completed', 'Successful', 'Paid') THEN
        INSERT INTO funnel_events (customer_id, session_id, product_id,
event_stage, event_timestamp, order_id)
        SELECT o.customer_id,
               (SELECT us.session_id FROM user_sessions us
                 WHERE us.customer_id = o.customer_id ORDER BY us.session_id
DESC LIMIT 1),
               oi.product_id, 'PAYMENT', COALESCE(NEW.payment_date, NOW()),
o.order_id
        FROM orders o JOIN order_items oi ON oi.order_id = o.order_id
        WHERE o.order_id = NEW.order_id;
    END IF;
END
```

View: cumulative funnel per product (`database/08_cumulative_funnel.sql`)

```
CREATE OR REPLACE VIEW vw_customer_product_reach AS
SELECT customer_id, product_id,
       MAX(FIELD(event_stage, 'VIEW','CART','CHECKOUT','ORDER','PAYMENT')) AS
max_rank,
       MAX(event_stage = 'REVIEW') AS reviewed
FROM funnel_events
WHERE product_id IS NOT NULL
GROUP BY customer_id, product_id;

CREATE OR REPLACE VIEW vw_product_funnel AS
SELECT p.product_id, p.product_title, p.brand,
       SUM(r.max_rank >= 1) AS viewed,   SUM(r.max_rank >= 2) AS carted,
       SUM(r.max_rank >= 3) AS checked_out, SUM(r.max_rank >= 4) AS ordered,
       SUM(r.max_rank >= 5) AS paid,     SUM(r.reviewed)      AS reviewed,
       ROUND(100 * SUM(r.max_rank >= 5) / NULLIF(SUM(r.max_rank >= 1), 0), 1)
AS view_to_paid_pct
FROM vw_customer_product_reach r
JOIN products p ON p.product_id = r.product_id
GROUP BY p.product_id, p.product_title, p.brand;
```

Analysis queries.

```
-- 1. Where are shoppers lost? Stages ordered by the share kept from the
previous stage
SELECT stage, customers, pct_from_previous
FROM vw_funnel_summary
```

```
WHERE pct_from_previous IS NOT NULL
ORDER BY pct_from_previous;

-- 2. Mobile against desktop: share of carts that became orders
SELECT device_type, sessions, carted, ordered, cart_to_order_pct
FROM vw_device_funnel
ORDER BY cart_to_order_pct;

-- 3. Money left in abandoned carts, by category (join + group + having)
SELECT p.category, COUNT(*) AS abandoned_items, SUM(a.price) AS value_lost
FROM vw_cart_abandonment a
JOIN products p ON p.product_id = a.product_id
GROUP BY p.category
HAVING COUNT(*) >= 5
ORDER BY value_lost DESC;

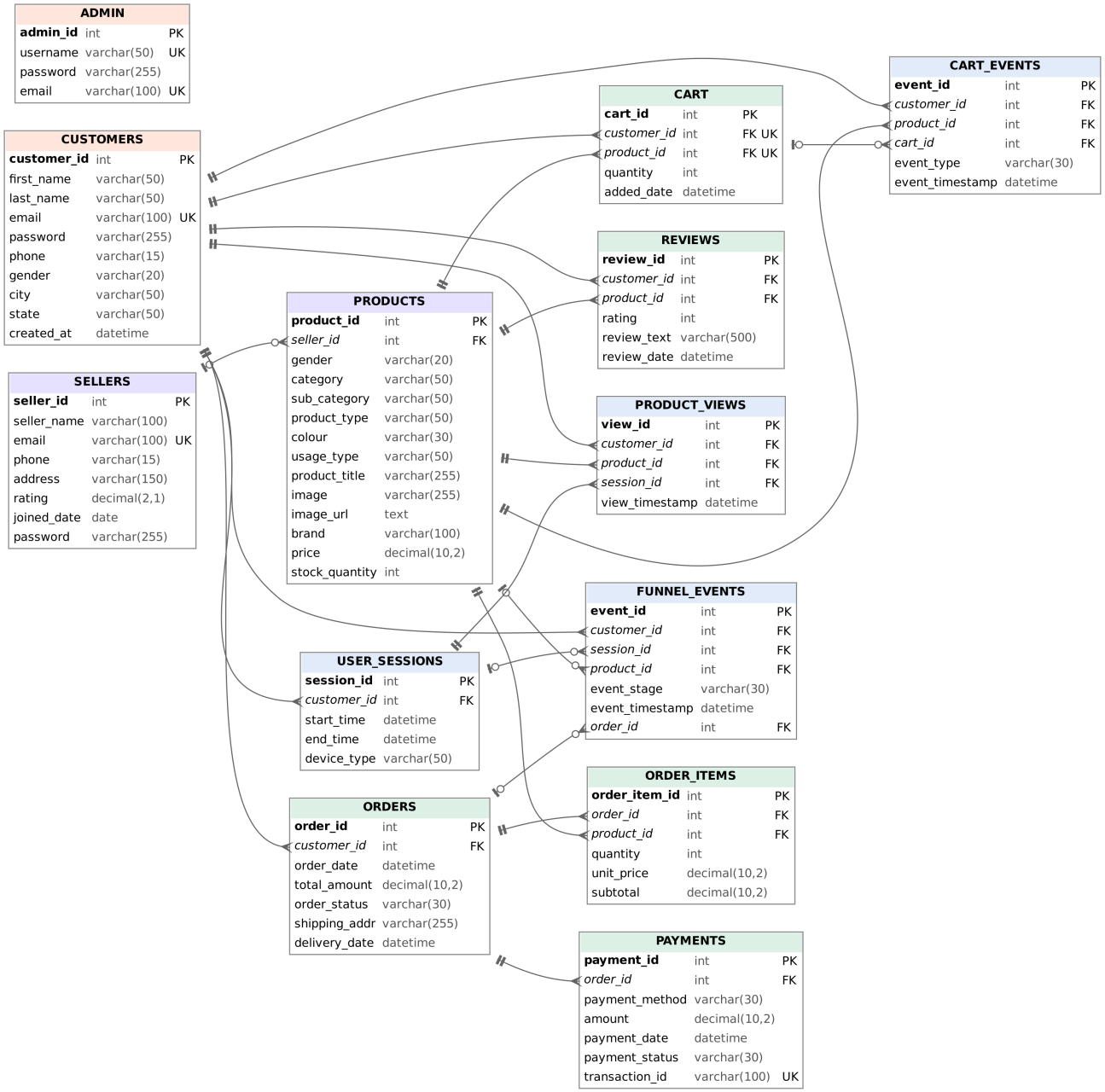
-- 4. Sellers earning more than the average seller (subquery)
SELECT seller_name, revenue
FROM vw_seller_sales
WHERE revenue > (SELECT AVG(revenue) FROM vw_seller_sales)
ORDER BY revenue DESC;

-- 5. Each customer's spending rank (window function)
SELECT c.first_name, c.last_name, SUM(o.total_amount) AS spent,
       RANK() OVER (ORDER BY SUM(o.total_amount) DESC) AS spend_rank
FROM customers c
JOIN orders o ON o.customer_id = c.customer_id
WHERE o.order_status IN ('Paid', 'Shipped', 'Delivered')
GROUP BY c.customer_id, c.first_name, c.last_name
ORDER BY spend_rank
LIMIT 10;
```

7. ER Diagram and UML Diagram

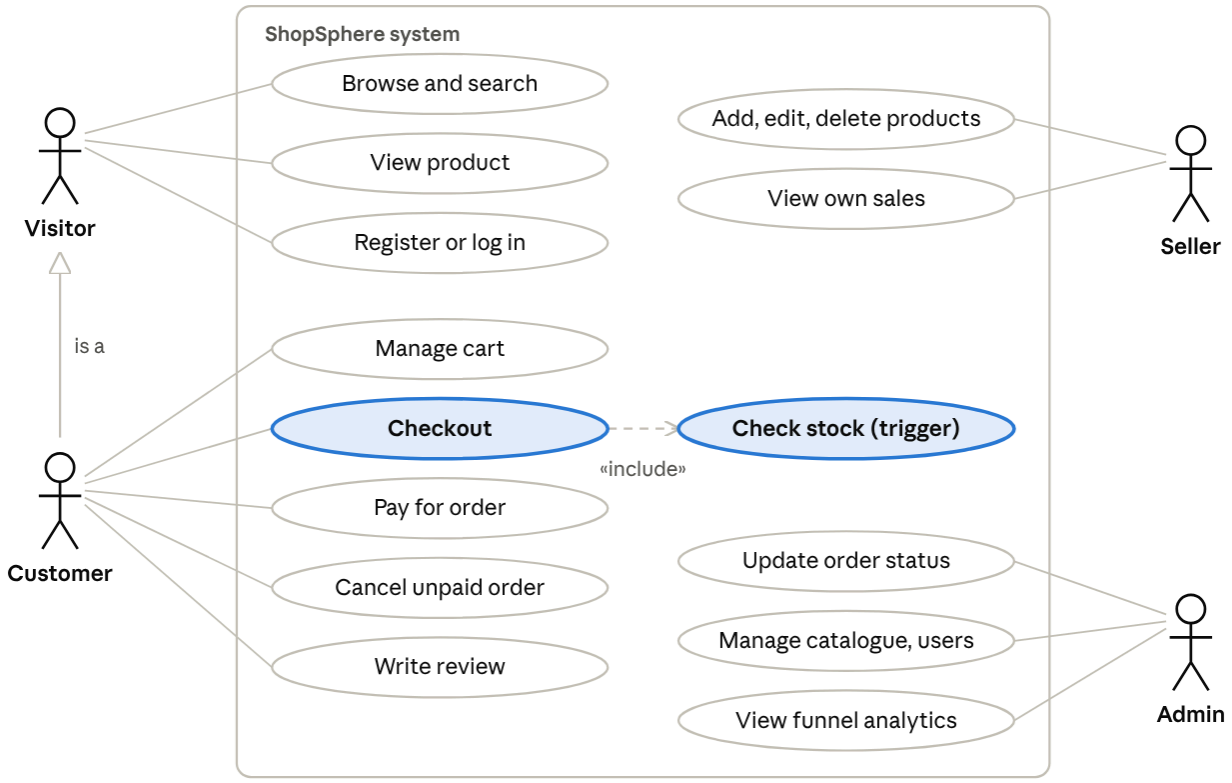
ER diagram

The ER diagram shows all 13 tables with their primary keys (PK), foreign keys (FK) and unique keys (UK). Crow's-foot notation: the three-pronged end marks the "many" side, a double bar marks "exactly one", and a circle marks "optional" (a nullable foreign key). Primary keys are bold, foreign keys italic. The full-resolution image is in the project at docs/er_diagram.png.


ADMIN
admin_id int PK
username varchar(50) UK
password varchar(255)
email varchar(100) UK
CUSTOMERS
customer_id int PK
first_name varchar(50)
last_name varchar(50)
email varchar(100) UK
password varchar(255)
phone varchar(15)
gender varchar(20)
city varchar(50)
state varchar(50)
created_at datetime
SELLERS
seller_id int PK
seller_name varchar(100)
email varchar(100) UK
phone varchar(15)
address varchar(150)
rating decimal(2,1)
joined_date date
password varchar(255)
CART
cart_id int PK
customer_id int FK UK
product_id int FK UK
quantity int
added_date datetime
CART_EVENTS
event_id int PK
customer_id int FK
product_id int FK
cart_id int FK
event_type varchar(30)
event_timestamp datetime
REVIEWS
review_id int PK
customer_id int FK
product_id int FK
rating int
review_text varchar(500)
review_date datetime
PRODUCTS
product_id int PK
seller_id int FK
gender varchar(20)
category varchar(50)
sub_category varchar(50)
product_type varchar(50)
colour varchar(30)
usage_type varchar(50)
product_title varchar(255)
image varchar(255)
image_url text
brand varchar(100)
price decimal(10,2)
stock_quantity int
PRODUCT_VIEWS
view_id int PK
customer_id int FK
product_id int FK
session_id int FK
view_timestamp datetime
FUNNEL_EVENTS
event_id int PK
customer_id int FK
session_id int FK
product_id int FK
event_stage varchar(30)
event_timestamp datetime
order_id int FK
USER_SESSIONS
session_id int PK
customer_id int FK
start_time datetime
end_time datetime
device_type varchar(50)
ORDER_ITEMS
order_item_id int PK
order_id int FK
product_id int FK
quantity int
unit_price decimal(10,2)
subtotal decimal(10,2)
ORDERS
order_id int PK
customer_id int FK
order_date datetime
total_amount decimal(10,2)
order_status varchar(30)
shipping_addr varchar(255)
delivery_date datetime
PAYMENTS
payment_id int PK
order_id int FK
payment_method varchar(30)
amount decimal(10,2)
payment_date datetime
payment_status varchar(30)
transaction_id varchar(100) UK

UML use-case diagram

Three user roles share one database; checkout always runs the stock trigger



UML use-case diagram · 4 actors, 14 use cases

A Customer is a Visitor who has logged in, so the customer inherits browsing and search. Checkout always includes the stock check, which runs inside MySQL as a trigger rather than in the web app.

8. Relational Schema

The schema has 13 relations. **Bold** marks the primary key and *italic* marks a foreign key; each foreign key's target is listed under its relation. Alternate keys (UNIQUE): SELLERS.email, CUSTOMERS.email, ADMIN.username, ADMIN.email, CART(customer_id, product_id), PAYMENTS.transaction_id.

- **SELLERS**(**seller_id**, seller_name, email, phone, address, rating, joined_date, password)
- **PRODUCTS**(**product_id**, *seller_id*, gender, category, sub_category, product_type, colour, usage_type, product_title, image, image_url, brand, price, stock_quantity) references: *seller_id* → SELLERS
- **CUSTOMERS**(**customer_id**, first_name, last_name, email, password, phone, gender, city, state, created_at)
- **ADMIN**(**admin_id**, username, password, email)

- **USER_SESSIONS**(**session_id**, *customer_id*, start_time, end_time, device_type)
  references: *customer_id* → CUSTOMERS
- **PRODUCT_VIEWS**(**view_id**, *customer_id*, *product_id*, *session_id*, view_timestamp)
  references: *customer_id* → CUSTOMERS; *product_id* → PRODUCTS; *session_id* → USER_SESSIONS
- **CART**(**cart_id**, *customer_id*, *product_id*, quantity, added_date)
  references: *customer_id* → CUSTOMERS; *product_id* → PRODUCTS
- **CART_EVENTS**(**event_id**, *customer_id*, *product_id*, *cart_id*, event_type, event_timestamp)
  references: *customer_id* → CUSTOMERS; *product_id* → PRODUCTS; *cart_id* → CART (ON DELETE SET NULL)
- **ORDERS**(**order_id**, *customer_id*, order_date, total_amount, order_status, shipping_addr, delivery_date)
  references: *customer_id* → CUSTOMERS
- **ORDER_ITEMS**(**order_item_id**, *order_id*, *product_id*, quantity, unit_price, subtotal)
  references: *order_id* → ORDERS; *product_id* → PRODUCTS
- **PAYMENTS**(**payment_id**, *order_id*, payment_method, amount, payment_date, payment_status, transaction_id)
  references: *order_id* → ORDERS
- **REVIEWS**(**review_id**, *customer_id*, *product_id*, rating, review_text, review_date)
  references: *customer_id* → CUSTOMERS; *product_id* → PRODUCTS
- **FUNNEL_EVENTS**(**event_id**, *customer_id*, *session_id*, *product_id*, event_stage, event_timestamp, *order_id*)
  references: *customer_id* → CUSTOMERS; *session_id* → USER_SESSIONS, *product_id* → PRODUCTS, *order_id* → ORDERS (all three ON DELETE SET NULL)

9. Database Tables

9.1 sellers

Attribute	Type	Null	Key	Default / extra
seller_id	INT	no	PK	—
seller_name	VARCHAR(100)	no	—	—
email	VARCHAR(100)	no	UNIQUE	—
phone	VARCHAR(15)	yes	—	—
address	VARCHAR(150)	yes	—	—

Attribute	Type	Null	Key	Default / extra
rating	DECIMAL(2,1)	yes	—	—
joined_date	DATE	yes	—	—
password	VARCHAR(255)	yes	—	—

CHECK constraints: `chk_sellers_email`, `chk_sellers_rating`.

Sample records:

seller_id	seller_name	email	phone	address	rating	joined_date	password
1	TrendStyle Fashion	trendstyle@gmail.com	9876543210	Bengaluru, Karnataka	4.8	2025-01-10	pbkdf2_sha256$...
2	Urban Wear Hub	urbanwear@gmail.com	9876543211	Chennai, Tamil Nadu	4.6	2025-01-15	NULL
3	Elite Apparel	eliteapparel@gmail.com	9876543212	Hyderabad, Telangana	4.7	2025-01-20	NULL

9.2 products

Attribute	Type	Null	Key	Default / extra
product_id	INT	no	PK	—
seller_id	INT	yes	FK → sellers(seller_id)	—
gender	VARCHAR(20)	yes	—	—
category	VARCHAR(50)	yes	—	—
sub_category	VARCHAR(50)	yes	—	—
product_type	VARCHAR(50)	yes	—	—
colour	VARCHAR(30)	yes	—	—
usage_type	VARCHAR(50)	yes	—	—
product_title	VARCHAR(255)	yes	—	—
image	VARCHAR(255)	yes	—	—
image_url	TEXT	yes	—	—
brand	VARCHAR(100)	yes	—	—
price	DECIMAL(10,2)	yes	—	—

Attribute	Type	Null	Key	Default / extra
stock_quantity	INT	yes	—	—

CHECK constraints: `chk_products_price`, `chk_products_stock`.

Sample records (key columns):

product_id	seller_id	gender	category	product_type	product_title	brand	price	stock_quantity
1636	10	Men	Footwear	Sports Shoes	Nike Men Air Zoom Century Shoes	Nike	3701.00	60
10096	12	Women	Footwear	Sports Shoes	Reebok Women Reeflex Run Blue Sports Shoes	Reebok	1834.00	142
12844	2	Boys	Apparel	Tshirts	Ant Kids Boys Multicoloured Striped T-shirts	Ant	4696.00	19

9.3 customers

Attribute	Type	Null	Key	Default / extra
customer_id	INT	no	PK	AUTO_INCREMENT
first_name	VARCHAR(50)	no	—	—
last_name	VARCHAR(50)	no	—	—
email	VARCHAR(100)	no	UNIQUE	—
password	VARCHAR(255)	no	—	—
phone	VARCHAR(15)	yes	—	—
gender	VARCHAR(20)	yes	—	—
city	VARCHAR(50)	yes	—	—
state	VARCHAR(50)	yes	—	—
created_at	DATETIME	yes	—	CURRENT_TIMESTAMP

CHECK constraints: `chk_customers_email`, `chk_customers_gender`.

Sample records (key columns):

customer_id	first_name	last_name	email	gender	city	state	created_at
1	Sai	Pillai	sai.pillai1@demo.shop	Other	Kochi	Kerala	2026-09-25

customer_id	first_name	last_name	email	gender	city	state	created_at
2	Aarav	Iyer	aarav.iyer2@demo.shop	Female	Chennai	Tamil Nadu	2026-09-26
3	Divya	Das	divya.das3@demo.shop	Male	Coimbatore	Tamil Nadu	2026-09-30

9.4 admin

Attribute	Type	Null	Key	Default / extra
admin_id	INT	no	PK	AUTO_INCREMENT
username	VARCHAR(50)	no	UNIQUE	—
password	VARCHAR(255)	no	—	—
email	VARCHAR(100)	no	UNIQUE	—

Sample record:

admin_id	username	password	email
1	admin	pbkdf2_sha256$…	admin@example.com

9.5 user_sessions

Attribute	Type	Null	Key	Default / extra
session_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—
start_time	DATETIME	yes	—	CURRENT_TIMESTAMP
end_time	DATETIME	yes	—	set at logout
device_type	VARCHAR(50)	yes	—	—

Sample records:

session_id	customer_id	start_time	end_time	device_type
1	1	2026-09-25 11:48	2026-09-25 12:04	Mobile
2	2	2026-09-26 11:37	2026-09-26 11:55	Tablet

session_id	customer_id	start_time	end_time	device_type
3	2	2026-09-28 22:18	2026-09-28 22:44	Desktop

9.6 product_views

Attribute	Type	Null	Key	Default / extra
view_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—
product_id	INT	no	FK → products(product_id)	—
session_id	INT	no	FK → user_sessions(session_id)	—
view_timestamp	DATETIME	yes	—	CURRENT_TIMESTAMP

Sample records:

view_id	customer_id	product_id	session_id	view_timestamp
2427	201	6628	410	2026-10-14 12:34
2428	201	24260	410	2026-10-14 12:34
2429	201	3567	410	2026-10-14 12:34

9.7 cart

Attribute	Type	Null	Key	Default / extra
cart_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id); UNIQUE with product_id	—
product_id	INT	no	FK → products(product_id); UNIQUE with customer_id	—
quantity	INT	no	—	—
added_date	DATETIME	yes	—	CURRENT_TIMESTAMP

CHECK constraints: `chk_cart_quantity`. Unique key: `uq_cart_customer_product`.

Sample records:

cart_id	customer_id	product_id	quantity	added_date
257	197	43883	1	2026-09-16 14:49
258	198	28606	1	2026-10-03 03:04
259	198	34858	1	2026-10-03 03:07

9.8 cart_events

Attribute	Type	Null	Key	Default / extra
event_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—
product_id	INT	no	FK → products(product_id)	—
cart_id	INT	yes	FK → cart(cart_id), ON DELETE SET NULL	—
event_type	VARCHAR(30)	no	—	—
event_timestamp	DATETIME	yes	—	CURRENT_TIMESTAMP

CHECK constraints: `chk_cart_event_type` (ADD, INCREASE, DECREASE, REMOVE).

Sample records (cart_id is NULL because these cart rows were later checked out or removed):

event_id	customer_id	product_id	cart_id	event_type	event_timestamp
270	201	1637	NULL	INCREASE	2026-10-14 08:43
271	201	1637	NULL	DECREASE	2026-10-14 08:43
272	201	1637	NULL	REMOVE	2026-10-14 08:43
281	201	10096	NULL	ADD	2026-10-14 12:33

9.9 orders

Attribute	Type	Null	Key	Default / extra
order_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—

Attribute	Type	Null	Key	Default / extra
order_date	DATETIME	yes	—	CURRENT_TIMESTAMP
total_amount	DECIMAL(10,2)	no	—	—
order_status	VARCHAR(30)	yes	—	Pending
shipping_addr	VARCHAR(255)	yes	—	—
delivery_date	DATETIME	yes	—	—

CHECK constraints: `chk_orders_status`, `chk_orders_total`.

Sample records:

order_id	customer_id	order_date	total_amount	order_status	shipping_addr	delivery_date
1	3	2026-09-30 20:39	6974.00	Paid	84, Main Road, Coimbatore, Tamil Nadu	2026-10-14
2	10	2026-10-07 22:44	6299.00	Paid	65, Main Road, Coimbatore, Tamil Nadu	2026-10-12
3	13	2026-09-19 05:21	7468.00	Paid	117, Main Road, Ahmedabad, Gujarat	2026-09-23

9.10 order_items

Attribute	Type	Null	Key	Default / extra
order_item_id	INT	no	PK	AUTO_INCREMENT
order_id	INT	no	FK → orders(order_id)	—
product_id	INT	no	FK → products(product_id)	—
quantity	INT	no	—	—
unit_price	DECIMAL(10,2)	no	—	—
subtotal	DECIMAL(10,2)	no	—	—

CHECK constraints: `chk_items_price`, `chk_items_quantity`, `chk_items_subtotal`.
Triggers: `trg_order_items_before_insert`, `trg_order_items_after_insert`.

Sample records:

order_item_id	order_id	product_id	quantity	unit_price	subtotal
1	1	31160	2	3487.00	6974.00
2	2	36957	2	2830.00	5660.00
3	2	50776	1	639.00	639.00

9.11 payments

Attribute	Type	Null	Key	Default / extra
payment_id	INT	no	PK	AUTO_INCREMENT
order_id	INT	no	FK → orders(order_id)	—
payment_method	VARCHAR(30)	no	—	—
amount	DECIMAL(10,2)	no	—	—
payment_date	DATETIME	yes	—	CURRENT_TIMESTAMP
payment_status	VARCHAR(30)	yes	—	Pending
transaction_id	VARCHAR(100)	yes	UNIQUE	—

CHECK constraints: `chk_payments_amount`, `chk_payments_status`. Trigger: `trg_payments_after_insert`.

Sample records:

payment_id	order_id	payment_method	amount	payment_date	payment_status	transaction_id
1	1	COD	6974.00	2026-09-30 20:41	Completed	DEMO1506637
2	2	Card	6299.00	2026-10-07 22:45	Completed	DEMO2154302
3	3	NetBanking	7468.00	2026-09-19 05:21	Completed	DEMO3691194

9.12 reviews

Attribute	Type	Null	Key	Default / extra
review_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—
product_id	INT	no	FK → products(product_id)	—

Attribute	Type	Null	Key	Default / extra
rating	INT	no	—	—
review_text	VARCHAR(500)	yes	—	—
review_date	DATETIME	yes	—	CURRENT_TIMESTAMP

CHECK constraints: `chk_review_rating` (1 to 5). Trigger: `trg_reviews_after_insert`.

Sample records:

review_id	customer_id	product_id	rating	review_text	review_date
1	13	26598	4	Nice, but size runs slightly small.	2026-09-28
2	14	40136	3	Average. Okay for daily use.	2026-10-01
3	19	5911	5	Great product, fast delivery.	2026-09-30

9.13 funnel_events

Attribute	Type	Null	Key	Default / extra
event_id	INT	no	PK	AUTO_INCREMENT
customer_id	INT	no	FK → customers(customer_id)	—
session_id	INT	yes	FK → user_sessions(session_id), ON DELETE SET NULL	—
product_id	INT	yes	FK → products(product_id), ON DELETE SET NULL	—
event_stage	VARCHAR(30)	no	index	—
event_timestamp	DATETIME	no	index	CURRENT_TIMESTAMP
order_id	INT	yes	FK → orders(order_id), ON DELETE SET NULL	—

CHECK constraints: `chk_funnel_stage` (VIEW, CART, CHECKOUT, ORDER, PAYMENT, REVIEW). Indexes: (event_stage, customer_id), (product_id, event_stage), (customer_id, event_timestamp), (event_timestamp).

Sample records: one demo customer's complete journey for one product, from viewing to reviewing:

event_id	customer_id	session_id	product_id	event_stage	event_timestamp	order_id
160	13	27	26598	VIEW	2026-09-19 05:07	NULL

event_id	customer_id	session_id	product_id	event_stage	event_timestamp	order_id
161	13	27	26598	CART	2026-09-19 05:08	NULL
165	13	27	26598	CHECKOUT	2026-09-19 05:21	NULL
168	13	27	26598	ORDER	2026-09-19 05:21	3
171	13	27	26598	PAYMENT	2026-09-19 05:21	3
173	13	27	26598	REVIEW	2026-09-28 05:21	NULL

10. Conclusion and Future Scope

Limitations.

- Payment is simulated; no money moves.
- Views and cart adds are recorded only for logged-in customers, because the funnel needs to know who the shopper is.
- The demo shoppers are generated, so their percentages show how the reports work rather than real market behaviour.

Future work. Guest carts that merge into the account on login, real payment gateway, email notifications on shipping, product photo upload by sellers, and indexes tuned with `EXPLAIN` on larger data.

Conclusion. ShopSphere shows how a database can do more than store data: constraints refuse bad values, triggers manage stock and record funnel stages, transactions keep orders whole, and views and stored procedures turn raw events into decisions. On the demo data, the clearest decision is to fix mobile checkout, where carts convert about 16 points worse than on desktop.